

Reproducing FORCE Learning in Chaotic Recurrent Neural Networks

Neiming Yin

June 11, 2026

Abstract

This project reproduces a compact result from Sussillo and Abbott [3]: a chaotic recurrent rate network trained with FORCE learning to autonomously generate a periodic signal composed of four sinusoids. The main implementation follows the external feedback architecture in Figure 1A and the Figure 2D target function from the paper and its supplemental Matlab script. The reproduction code separates configuration, target generation, recursive least-squares updates, metrics, plotting, and tests so that the workflow is transparent and repeatable. A small all-to-all internal-learning extension is also included as a second-stage experiment.

Background and Introduction

Recurrent neural networks can produce rich spontaneous dynamics because activity is continuously fed back through recurrent synapses. When the recurrent gain is large, random rate networks can become chaotic [2]. This creates a useful reservoir of temporal activity, but it also makes direct control difficult.

Sussillo and Abbott [3] proposed FORCE learning as a way to turn chaotic spontaneous activity into coherent target patterns. The algorithm rapidly suppresses output error during learning and then leaves a stable set of weights that can autonomously regenerate the desired signal. The paper demonstrates many outputs, including periodic signals, noisy targets, discontinuous functions, Lorenz-like trajectories, and movement patterns.

This reproduction focuses on the most direct starting point: Figure 2D, a periodic function composed of four sinusoids, trained in the Figure 1A external-feedback architecture. This choice is appropriate for a first reproduction because the paper's supplemental Matlab code contains a compact reference implementation, and the target is simple enough to inspect while still exercising the core FORCE/RLS mechanism.

Methods

The model

The primary experiment uses the external feedback architecture shown in Figure 1A of Sussillo and Abbott [3]. The model consists of a recurrent generator network, a linear readout unit, and a feedback pathway from the readout output back to the recurrent network. The recurrent network contains N rate units. The internal state is denoted by $x(t)$, and the firing-rate vector is

$$r(t) = \tanh(x(t)).$$

The recurrent connectivity matrix M is randomly generated with connection probability p and scaling $g/\sqrt{pN}$. The network output is a linear readout of the firing rates:

$$z(t) = w^T r(t),$$

where w is the readout weight vector. During the Figure 1A experiment, only w is modified; the recurrent matrix M and the feedback weights w_f remain fixed.

The discrete-time dynamics used in the Matlab implementation are

$$x(t + \Delta t) = (1 - \Delta t)x(t) + \Delta t M r(t) + \Delta t w_f z(t).$$

The output $z(t)$ is fed back to the network, allowing the trained readout to influence future network state. This closed-loop structure enables autonomous generation after training.

Target function

The target is the Figure 2D signal from the paper and supplemental code:

$$f(t) = \frac{1}{1.5} \left[a \sin(\pi q t) + \frac{a}{2} \sin(2\pi q t) + \frac{a}{6} \sin(3\pi q t) + \frac{a}{3} \sin(4\pi q t) \right],$$

with $a = 1.3$ and $q = 1/60$ for the external-feedback experiment.

FORCE/RLS training

At each learning step, the instantaneous error is

$$e(t) = z(t) - f(t).$$

The readout vector is updated with recursive least squares:

$$k = Pr, \quad c = \frac{1}{1 + r^T k}, \quad P \leftarrow P - k k^T c, \quad w \leftarrow w - e k c.$$

The matrix P is initialized as I/α . In the code this update is isolated in `code/rls_update.m`; the full Figure 1A simulation is implemented in `code/run_force_external.m`.

Training and testing are separated. During training, w is updated every `learn_every` time steps. During testing, learning is stopped and the network continues from the final training state. The output is then compared with the target over the next time interval.

Algorithm summary

1. Initialize random network M , feedback w_f , readout $w = 0$, and $P = I/\alpha$.
2. For each training time step, update x , $r = \tanh(x)$, and $z = w^T r$.
3. At each learning step, compute $e = z - f(t)$, then update w and P with RLS.
4. Store the training output $z(t)$ and the readout norm $\|w\|_2$.
5. Stop learning and continue the network forward to record autonomous testing output.

Results

The codebase provides three levels of experiments. The test preset checks dimensions, numerical finiteness, RLS symmetry, and deterministic fixed-seed behavior. The smoke preset runs a small Figure 1A experiment and writes a trace figure and data file. The reproduction preset uses the larger reference scale $N = 1000$, $p = 0.1$, $g = 1.5$, $\alpha = 1$, $\Delta t = 0.1$, and 1440 seconds of simulated training time, matching the supplemental Matlab example.

The completed full reproduction produced training MAE = 0.005416, testing MAE = 0.036433, and testing correlation = 0.997139. The red network output closely overlaps the green target during the autonomous test phase, reproducing the qualitative behavior shown in Figure 2D of Sussillo and Abbott [3]. The full result is saved as `results/data/figure2d_external.mat`; the report figure is stored as `report/figures/force-traces.pdf`.

The parameter sweep script varies g and α . This is used to document tuning rather than to claim a new result. The expected pattern, consistent with the paper's discussion around Figure 5, is that strongly recurrent networks with $g > 1$ are useful for FORCE learning, while α controls how aggressive the early RLS updates are.

Conclusion

This project implements a reproducible MATLAB workflow for the Figure 2D FORCE learning result. The central mechanism is the RLS update that keeps the network output close to the target during training while finding a fixed readout vector that supports autonomous generation afterward.

The implementation is intentionally modular: the target function, RLS rule, simulation loop, metrics, plotting, tests, and scripts are separated. This makes the reproduction easier to inspect than the original single-file examples while preserving the same mathematical structure.

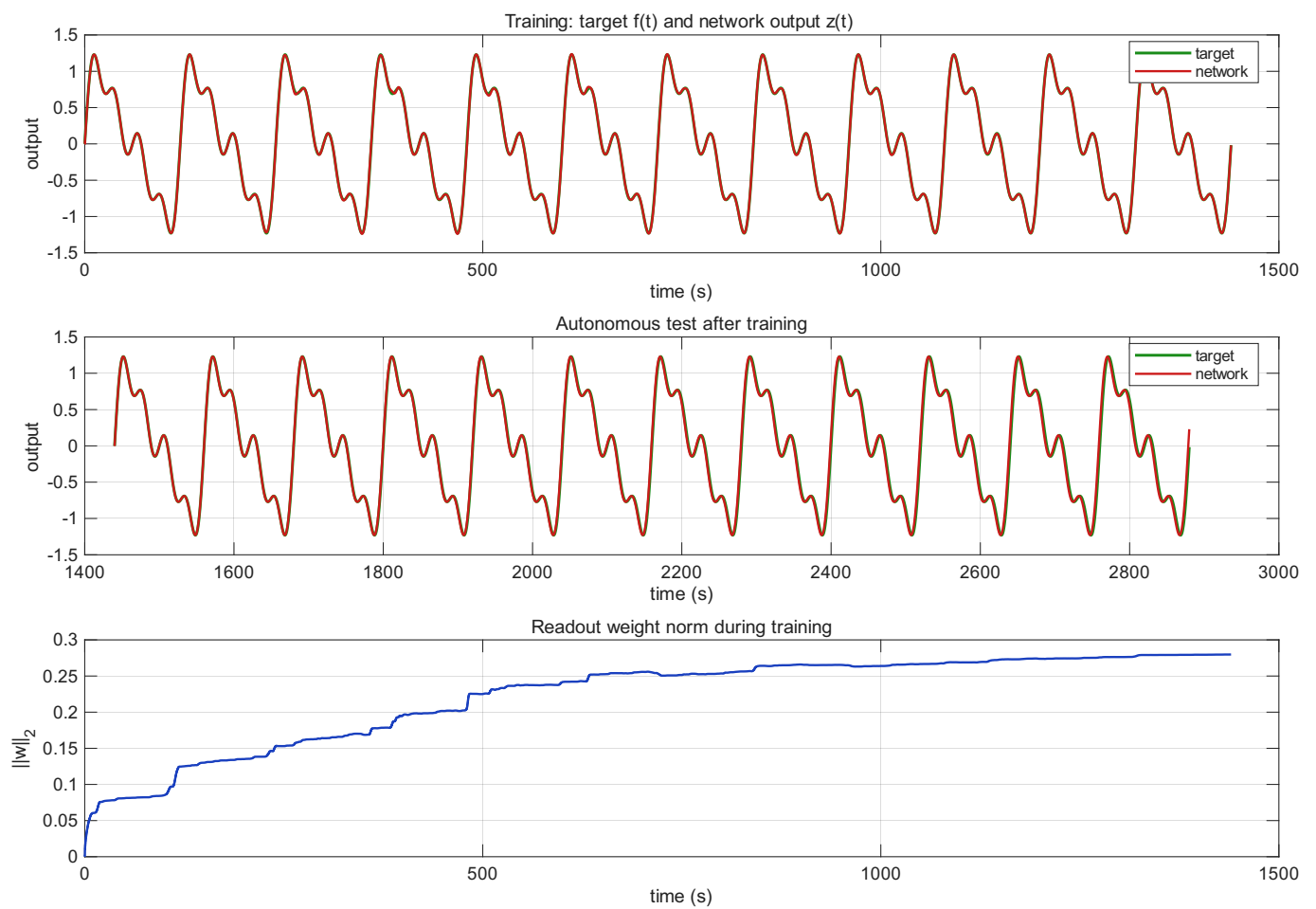


Figure 1: FORCE reproduction traces. The top panel shows training, the middle panel shows autonomous testing after weight updates stop, and the bottom panel shows the readout weight norm.

Future extensions include running the full Figure 1C internal-learning reproduction, adding the sparse internal-learning case, and reproducing additional targets such as the noisy four-sinusoid example, square wave, or Lorenz-like trajectory.

References

References

- [1] Dayan, P. and Abbott, L. F. (2005). *Theoretical Neuroscience: Computational and Mathematical Modeling of Neural Systems*. MIT Press.
- [2] Sompolinsky, H., Crisanti, A., and Sommers, H. J. (1988). Chaos in random neural networks. *Physical Review Letters*, 61(3), 259–262.
- [3] Sussillo, D. and Abbott, L. F. (2009). Generating coherent patterns of activity from chaotic neural networks. *Neuron*, 63(4), 544–557.
- [4] Haykin, S. (2002). *Adaptive Filter Theory*. Prentice Hall.
- [5] Jaeger, H. and Haas, H. (2004). Harnessing nonlinearity: predicting chaotic systems and saving energy in wireless communication. *Science*, 304(5667), 78–80.

Reading Notes

The detailed step-by-step workflow is maintained in `../docs/workflow.md`. The most important reading anchors are:

- Main paper page 3: Figure 2 and the Figure 2D four-sinusoid target.
- Main paper pages 4–6: model output $z = w^T r$, external feedback, and RLS/FORCE learning.
- Main paper page 10: internal learning in the generator network, corresponding to the Figure 1C extension.
- Supplemental Matlab scripts: `force_external_feedback_loop.m`, `force_internal_all2all.m`, and `force_internal_sparse.m`.